

STRING MATCHING ALGORITHMS

Module 3
BCSE204L – Design and Analysis of Algorithms



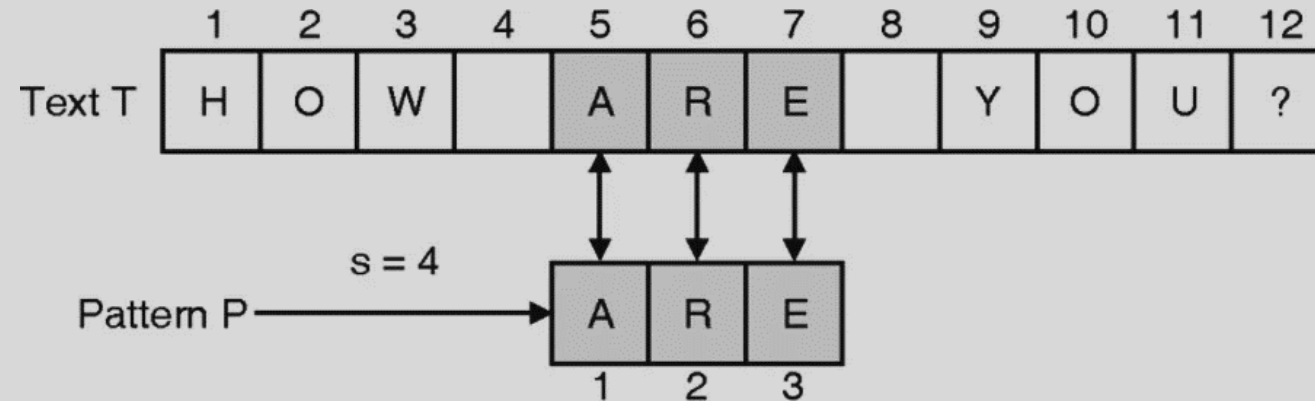
NAÏVE STRING MATCHING

String Matching Algorithm

- String matching operation is a core part in many text processing applications. The objective of this algorithm is to find pattern **P** from given text **T**. Typically $|P| \ll |T|$. In the design of compilers and text editors, string matching operation is crucial. So locating **P** in **T** efficiently is very important.
- The problem is defined as follows: “*Given some text string $T[1...n]$ of size n , find all occurrences of pattern $P[1...m]$ of size m in T .*”
- We say that **P** occurs in text **T** with number of shifts s , if $0 \leq s \leq n - m$ and $T[(s + 1) \dots (s + m)] = P[1...m]$.

String Matching Algorithm

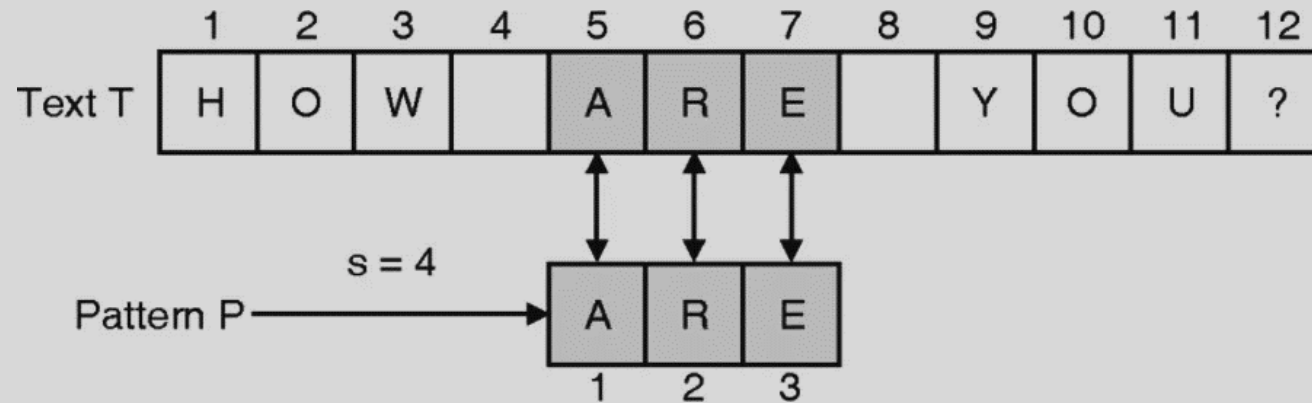
Consider the following example



- In this example, pattern $P = ARE$ is found in text T after four shifts.
- The classical application of such algorithms are to find particular protein pattern in DNA sequence.

String Matching Algorithm

Consider the following example



- Strings may be encoded using set of character alphabets {a, b, ..., z}, binary alphabets {0, 1}, decimal alphabets {0, 1, 2, ..., 9}, DNA alphabets {A, C, G, T}. The encoding of the string directly affects the efficiency of searching.

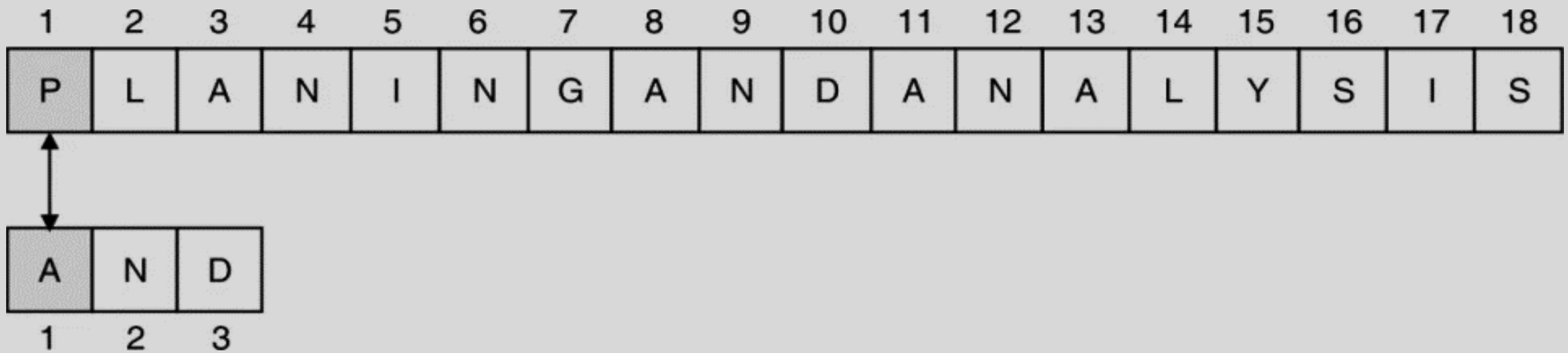
Naïve String Matching Algorithm

Working Mechanism

- This is simple and efficient brute force approach. It compares the first character of pattern with searchable text. If a match is found, pointers in both strings are advanced. If a match is not found, the pointer to text is incremented and pointer of the pattern is reset. This process is repeated till the end of the text.
- The naïve approach does not require any pre-processing. Given text **T** and pattern **P**, it directly starts comparing both strings character by character.
- After each comparison, it shifts pattern string *one position* to the right.
- Following example illustrates the working of naïve string matching algorithm. Here, **T = PLANINGANDANALYSIS** and **P = AND**
- Here, t_i and p_j are indices of text and pattern respectively.

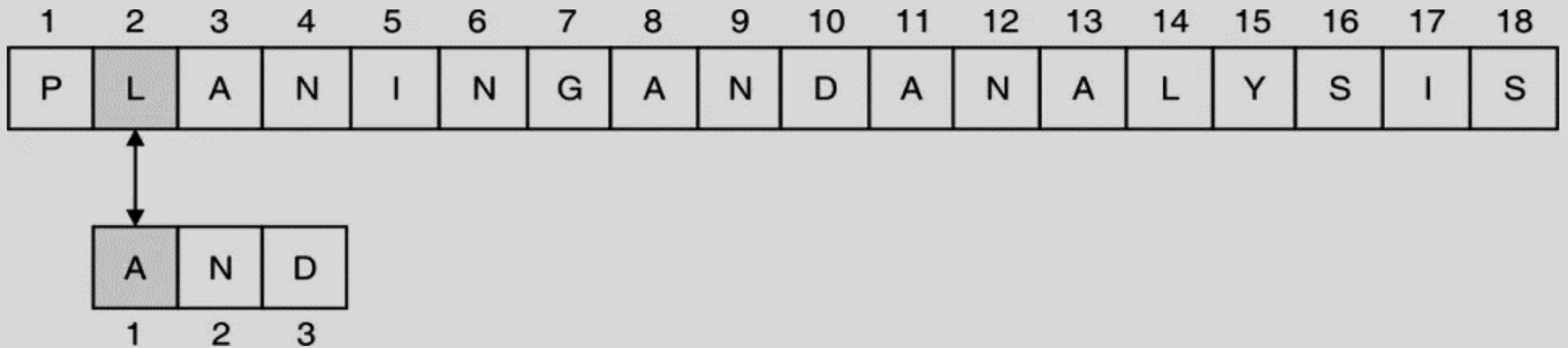
Naïve String Matching Algorithm

Step 1: $T[1] \neq P[1]$, so advance text pointer, i.e. t_i++



Naïve String Matching Algorithm

Step 2: $T[2] \neq P[1]$, so advance text pointer, i.e. t_i++



Naïve String Matching Algorithm

Step 3: $T[3] = P[1]$, so advance text pointer, i.e. t_i++ , p_j++



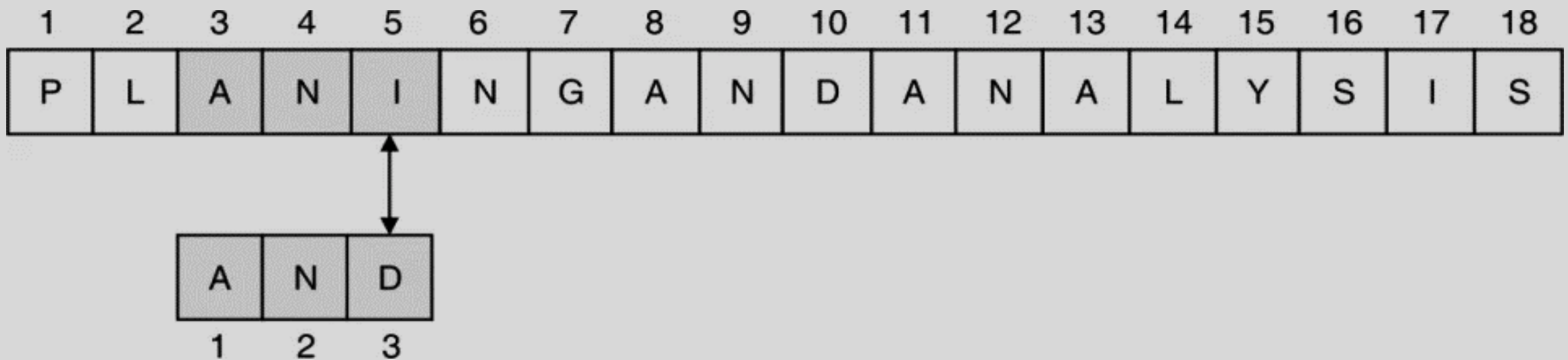
Naïve String Matching Algorithm

Step 4: $T[4] = P[2]$, so advance text pointer, i.e. t_i++ , p_j++



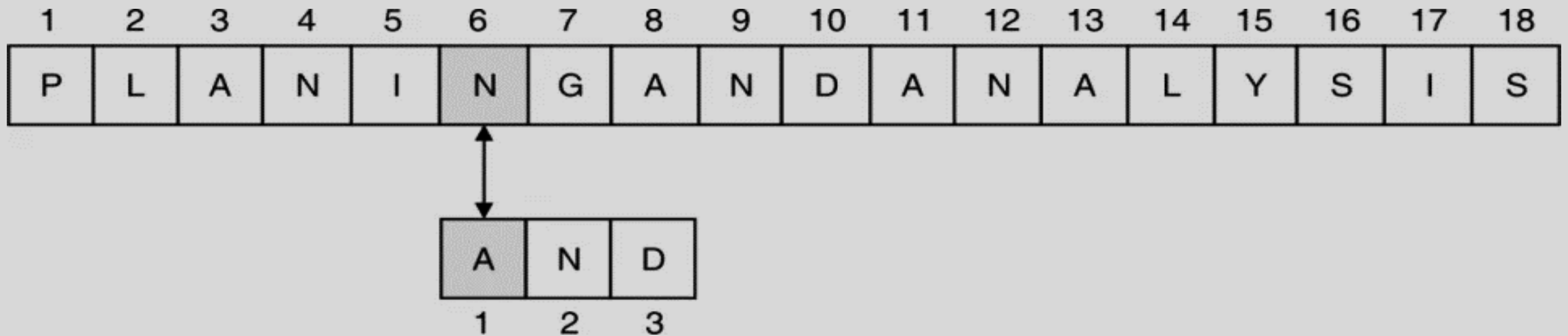
Naïve String Matching Algorithm

Step 5: $T[5] \neq P[3]$, so advance text pointer and reset pattern pointer, i.e. t_i++ , $p_j = 1$



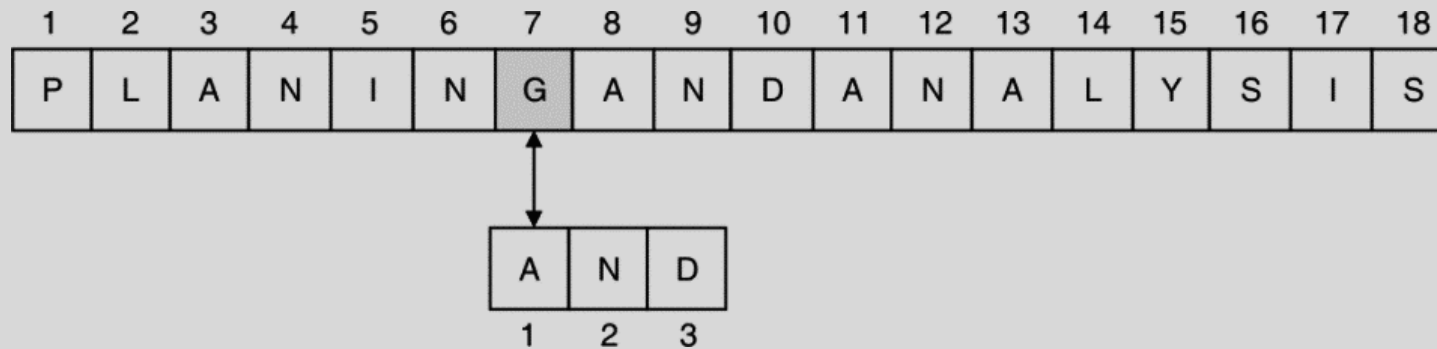
Naïve String Matching Algorithm

Step 6: $T[6] \neq P[1]$, so advance text pointer, i.e. t_i++ ,

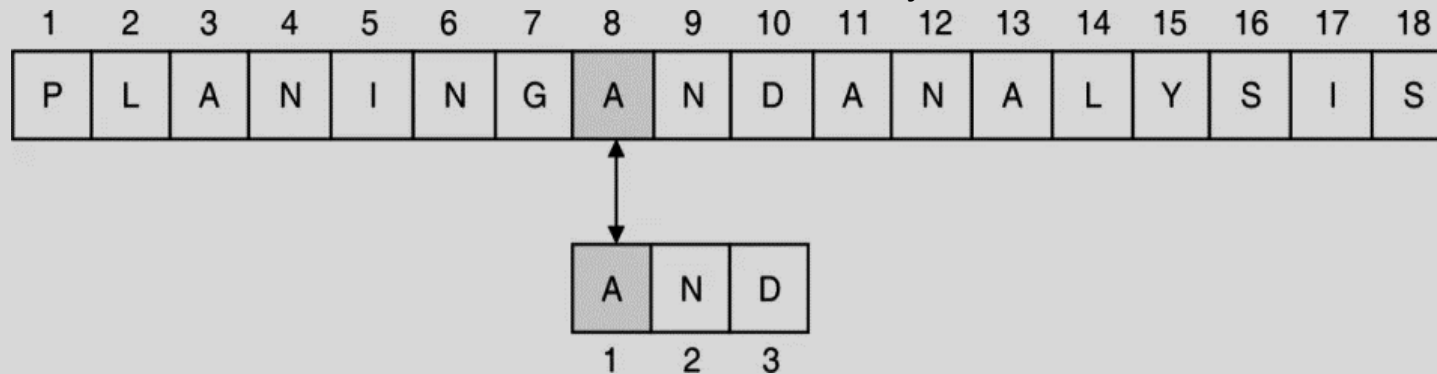


Naïve String Matching Algorithm

Step 7: $T[7] \neq P[1]$, so advance text pointer, i.e. t_i++ ,

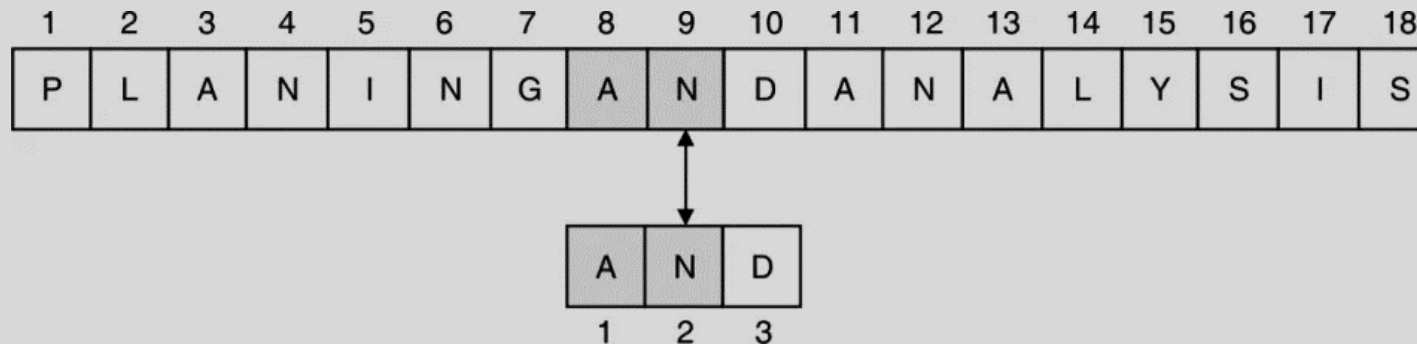


Step 8 : $T[8] = P[1]$, so advance both pointers, i.e. t_i++ , p_j++

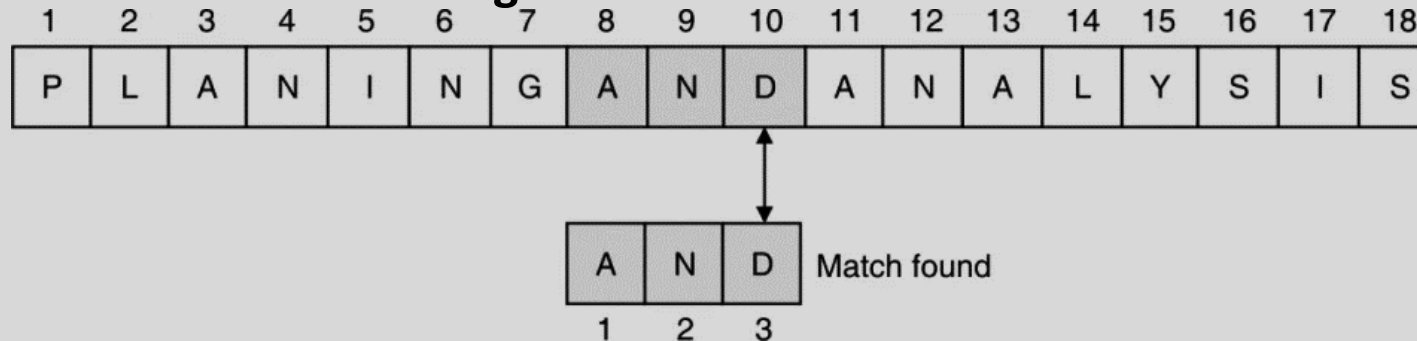


Naïve String Matching Algorithm

Step 9: $T[9] = P[2]$, so advance both pointers, i.e. $ti++$, $pj++$



Step 10 : $T[10] = P[3]$, so advance both pointers, i.e. $ti++$, $pj++$, process continues till the possible comparison in the text string.



Naïve String Matching Algorithm

Algorithm

- **NAÏVE_STRING_MATCHING(T, P)**
- // T is the text string of length n
- // P is the pattern of length m
- **FOR** $i \leftarrow 0$ to $n - m$ **DO**
- **IF** $P[1 \dots m] == T[i+1 \dots i+m]$ **THEN**
- print “Match Found”
- **END**
- **END**

Naïve String Matching Algorithm

Algorithm

- There are two cases of consideration :
 - **Pattern found**
 - The worst case occurs when the pattern is at last position and there are spurious hits all the way. Example, **T = AAAAAAAAAAB**, **P = AAAB**.
 - To move pattern one position right, m comparisons are made. Searchable text in **T** has a **length $(n - m)$** . Hence, in worst case algorithm runs in **$O(m * (n - m))$ time**.
 - **Pattern not found**
 - In the best case, the searchable text does not contain any of the prefixes of the pattern. Only one comparison requires moving pattern one position right.

Naïve String Matching Algorithm

Advantages:

- No Pre-processing phase required because the running time of Naive-String-Matcher is equal to its matching time.
- No extra space are needed.
- Also, the comparisons can be done in any order.

Disadvantages:

- Naive method is inefficient because information from a shift is not used again.



RABIN KARP ALGORITHM

Rabin-Karp String Matching Algorithm

Working Mechanism

- Comparing numbers is easier and cheaper than comparing strings. Rabin Karp algorithm represents strings in numbers.
- Suppose $\mathbf{p}$ represents values corresponding to pattern $\mathbf{P[1...m]}$ of length $\mathbf{m}$. And $\mathbf{t_s}$ represents values of $\mathbf{m}$ -length substrings $\mathbf{T[(s + 1) ... (s + m)]}$ for $\mathbf{s = 0, 1, 2, ..., n - m}$.
- We can compute $\mathbf{p}$ in $\mathbf{O(m)}$ time and all $\mathbf{ts}$ can be computed in $\mathbf{O(n - m + 1)}$ time.
- Rabin Karp algorithm is based on hashing technique. It first computes the hash value of $\mathbf{p}$ and $\mathbf{t_s}$.
- If hash values are same, i.e. if $\mathbf{hash(p) = hash(t_s)}$, we check the equality of inverse hash similar to a naïve method. If hash values are not same, no need to compare actual string.

Rabin-Karp String Matching Algorithm

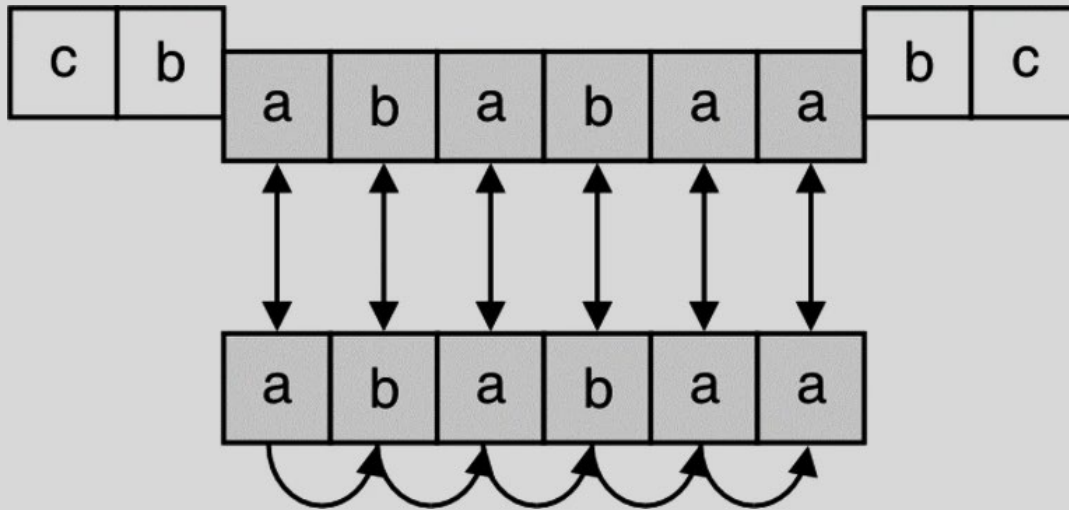
Working Mechanism

- On the hash match, actual characters of both strings are compared using brute force approach. If the pattern is found, then it is called ***hit***. Otherwise, it is called a ***spurious hit***.
- For example, let us consider the hash value of string **T = ABCDE** is 38 and hash of string **P = ABCDX** is **71**. Clearly, hash values are not same, so strings cannot be same. Brute force approach does five comparisons whereas Rabin Karp dose only one comparison.
- However, the same hash value does not ensure the string match. Two different strings can have same hash values. That is why we need to compare them character by character on hash hit.

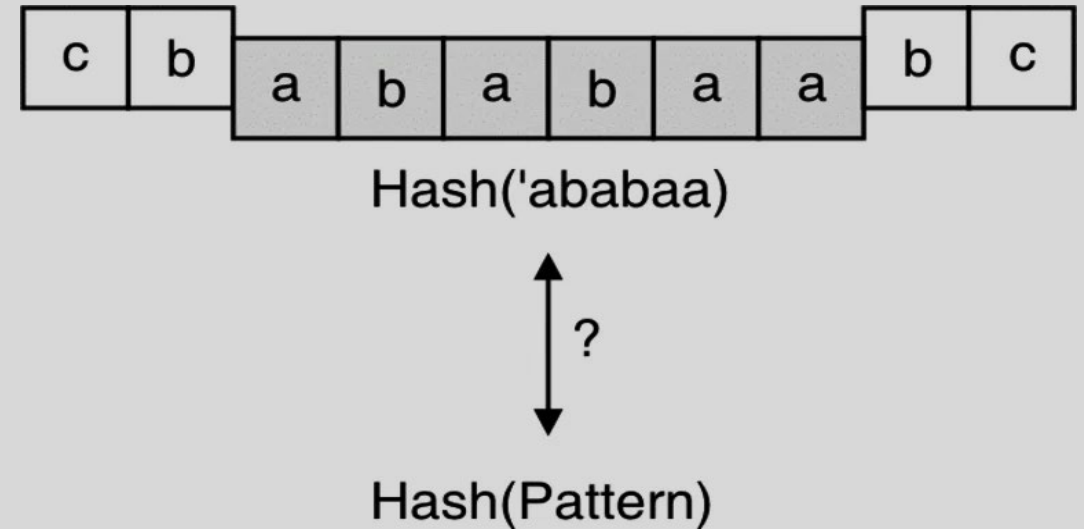
Rabin-Karp String Matching Algorithm

Difference between Brute Force and Rabin Karp approach

Brute Force Approach



Rabin-Karp Approach



Rabin-Karp String Matching Algorithm

- Given pattern **P[1...m]**, we can derive its numeric value **p** in base **d** in **O(m)** time as follow:

$$p = P[m] + d(P[m - 1] + d(P[m - 2] + \dots + d(P[2] + dP[1]) \dots))$$

- Similarly, we can derive numeric value of first substring **t₀** of length **m** from text **T[1...n]** in **O(m)** time. Remaining all **t_i**, **i = 1, 2, 3, ..., n - m**, can be derived in constant time.
- Given **t_s**, we can compute **t_{s+1}** as,

$$t_{s+1} = d(t_s - d^{m-1} T[s + 1]) + T[s + m + 1]$$

Rabin-Karp String Matching Algorithm

- Assume that $T = [4, 3, 1, 5, 6, 7, 5, 9, 3]$ and $P = [1, 5, 6]$. Here length of P is 3, so $m = 3$. Consider that, for given pattern P , its value $p = 156$, and $t_0 = 431$

$$t_1 = 10(431 - 10^2 T[1]) + T[4] = 10(431 - 400) + 5 = 315$$

- Values of p and t_s may be too large to process. We can reduce these values by taking its modulo with suitable number q . typically, q is a prime number.
- Mod function has some nice mathematical property.

$$[(a \bmod k) + (b \bmod k)] \bmod k = (a + b) \bmod k$$

$$(a \bmod k) \bmod k = a \bmod k$$

Rabin-Karp String Matching Algorithm

- Computing hash value of every subsequence of m character of text may turn out to be time-consuming. However, a bit of mathematics makes it easier.
- Suppose t_s represents decimal value of substring $T[(s + 1) \dots (s + m)]$. If hash of t_s is known, **hash value** can directly be derived for t_{s+1} as follow :
- Hash for $t_{s+1} = (d*(t_s - T[s + 1]h) + T[s + m + 1]) \bmod q$, where $h = d^{m-1} \bmod q$

Rabin-Karp String Matching Algorithm

- **Two facts :**
- **$t_s = p \bmod q$, does not mean $t_s = p$, $t_s \neq p \bmod q$, means $t_s \neq p$**
- **If $p = 45365$, $t_s = 64371$ and $q = 11$,**

$$45365 \bmod 13 = 8$$

$$64371 \bmod 13 = 8$$

- From above example, it is clear that two different strings might have same mod. We can reject all negative tests to rule out all possible invalid shifts. And all positive tests must be validated to overcome spurious hits.
- Spurious hits reduce with a larger value of q , and it increases with smaller q .

Rabin-Karp String Matching Algorithm

Algorithm

RABIN_KARP(T, P)

// T is text of length n

// P is pattern of length m

$h \leftarrow \text{power}(d, m - 1) \bmod q$

$p \leftarrow 0$

$t_0 \leftarrow 0$

FOR $i \leftarrow 1$ to m **DO**

$p \leftarrow (d * p + P[i]) \bmod q$

$t_0 \leftarrow ((d * t_0) + T[i]) \bmod q$

END

FOR $s \leftarrow 0$ to $n - m$ **DO**

IF $p == t_0$ **THEN**

IF $P[1...m] == T[s+1 \dots s + m]$ **THEN**

print "Match found at shift", s

END

END

IF $s < (n - m)$ **THEN**

$t_{s+1} \leftarrow (d * (t_s - T[s + 1]) * h) + T[s + m + 1]$

END

END

Rabin-Karp String Matching Algorithm

Complexity Analysis:

- Rabin Karp algorithm is a randomized algorithm. In most of the cases, it runs in linear time, i.e. in **$O(n)$** . However, the worst case of Rabin-Karp algorithm is as bad as a naïve algorithm, i.e. **$O(mn)$** , but it's rare.
- It can happen only when prime number used for hashing is very small.

Rabin-Karp String Matching Algorithm

Example: Explain spurious hits in Rabin-Karp string matching algorithm with an example. Working modulo $q = 13$, how many spurious hits does the Rabin-Karp matcher encounter in the text $T = 2359023141526739921$ when looking for the pattern $P = 31415$?

Rabin-Karp String Matching Algorithm

- **Solution:**

Given pattern $P = 31415$, and prime number $q = 13$

$$P \bmod q = 31415 \bmod 13 = 7$$

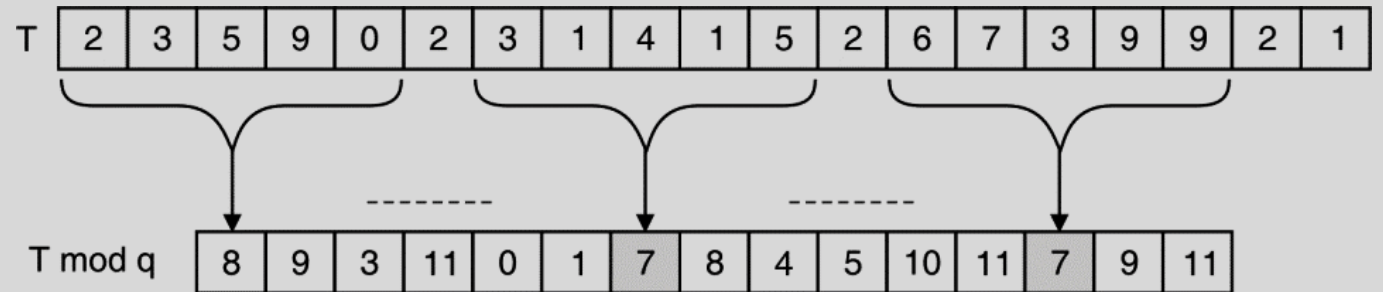
Let us find hash value for given text :

$$T[1\dots 5] = 23590 \bmod 13 = 8$$

$$T[2\dots 6] = 35902 \bmod 13 = 9$$

:

$$T[(m - 4) \dots m] = 39921 \bmod 13 = 11$$



Rabin-Karp String Matching Algorithm

- The hash value of pattern P is 7. We have two such values in a hash(T). So there may be a spurious hit or actual string. In given text T, one hit is an actual match and one is spurious hit.
- If the hash of pattern and any substring in the text is same, we have two possibilities :
- **Hit** : Pattern and text are same
- **Spurious hit** : Hash value is same but pattern and corresponding text is not same

Rabin-Karp String Matching Algorithm

- Here, m = length of pattern = 5.
- Consider
- $ts = 23590$. Next value $ts+1$ is derived as,
- $t_{s+1} = 10(t_s) - 10^{m \cdot T[s+1]} + T[s + m + 1]$
- $= (10 \cdot 23590) - 10^{5 \cdot 2} + 2$
- $= 235900 - 200000 + 2 = 35902$
- $t_{s+2} = 10(t_{s+1}) - 10^{m \cdot T[s+2]} + T[s + m + 2]$
- $= (10 \cdot 35902) - 10^{5 \cdot 3} + 3$
- $= 359020 - 300000 + 3 = 59023$

Rabin-Karp String Matching Algorithm

- In same way, we can compute the next t_{s+i} using incremental approach.
- Rabin Karp algorithm matches hash value, rather than directly comparing actual string value. If hash value of pattern **P** and the hash value of subsequence in string **T** are same, the actual value of strings is compared using brute force approach. Like t_{s+1} , we can also derive hash value incrementally as shown below.

Rabin-Karp String Matching Algorithm

- Calculation for hash of 14152
- If t_s is known, the hash value can directly be derived for t_{s+1} as follow.
- Hash for $t_{s+1} = (d*(t_s - T[s+1]h) + T[s + m + 1]) \bmod q$
- $= 10*(31415 - (3*10^4 \bmod 13)) + 2 \bmod 13$
- $= 10(31415 - 9) + 2 \bmod 13$
- $= 314062 \bmod 13 = 8$



KMP ALGORITHM

Knuth Morris Pratt (**KMP**) Algorithm

Introduction

- This algorithm is also known as **KMP (Knuth-Morris-Pratt) algorithm**. This is the first linear time algorithm for string matching. It utilizes the concept of a naïve approach in some different ways. This approach **keeps track of matched part of the pattern**.
- The main idea of the algorithm is to **avoid computation of transition function δ and reduce useless shifts performed in a naïve approach**.
- By maintaining the information of the already processed string, this approach reduces the time to **$O(m + n)$** . That is, in the worst case, the KMP algorithm examines all characters of the input pattern and text exactly once.

Knuth Morris Pratt (**KMP**) Algorithm

Introduction

- This improvement is achieved by using the auxiliary function π . The naïve approach was not storing anything it has already matched, and that's why it was shifting the pattern right side by just one. Computation of auxiliary function π takes $O(m)$ time.
- Function π is very useful, it computes the number of shifts of the pattern by finding pattern matches within itself.

Knuth Morris Pratt (**KMP**) Algorithm

Sliding Rule:

- The naïve approach slides the pattern by one right side. The KMP approach slides the string multiple steps and reduces comparisons. Let us consider the pattern

$$P = p_1 p_2 p_3 \dots p_{m-1} p_m.$$

- For any string **P**, sub-string **P'** of the **first i** characters is called the prefix of **P**, where **i = 1 to m**. Null string also belongs prefix of **P**. String **P'** is proper prefix of **P** if and only if **P' ≠ P**.
- Similarly, for any string **P**, substring **P'** of **last i** characters is called the suffix of **P**, where **i = 1 to m**. Null string also belongs suffix of **P**. String **P'** is the proper suffix of **P** if and only if **P' ≠ P**.

Knuth Morris Pratt (**KMP**) Algorithm

Sliding Rule:

- Consider the pattern **P** = **abcbdabc** ;
- Prefixes of **P** are : {**f**, **a**, **ab**, **abc**, **abcd**, **abcda**, **abcdab**, **abcdabc**}, where **abcdabc** is not the proper prefix of **P**.
- Suffixes of **P** are : {**f**, **c**, **bc**, **abc**, **dabc**, **cdabc**, **bcdabc**, **abcdabc**}, where **abcdabc** is not the proper suffix of **P**.
- Suppose the pattern **P[1....q]** is matched with the text at **T[(i - q + 1)... i]**. The first mismatch occurs at position **P[q + 1]** and **T[i + 1]**. Slide the pattern right by **x**, where **x** is the length of the longest proper prefix of **P[1...q]** and also the longest suffix of **P[1...q]**

Knuth Morris Pratt (**KMP**) Algorithm

Working Principle:

- If $P[1...q]$ matches with $T[(i - q + 1) ...i]$, there are two possibilities :
- If $P[q + 1] = T[i + 1]$, matched string length is increased by one. If $q = m$, a complete match is found and the algorithm terminates.
- If $P[q + 1] \neq T[i + 1]$, then the slide pattern P to right by longest common suffix and prefix of $P[1...q]$.

Knuth Morris Pratt (**KMP**) Algorithm

KMP Algorithm:

PREFIX_FUNCTION(P)

// P is the pattern of length m

$\pi[1] \leftarrow 0$

$k \leftarrow 0$

FOR $q \leftarrow 2$ to m **DO**

WHILE $k > 0$ and $P[k+1] \neq P[q]$ **DO**

$k \leftarrow \pi[k]$

END

IF $P[k+1] == P[q]$ **THEN**

$k \leftarrow k + 1$

END

$\pi[q] \leftarrow k$

END

RETURN π

Knuth Morris Pratt (**KMP**) Algorithm

KMP Algorithm:

KMP_STRING_MATCHING(T, P)

// T is text of length n

// P is pattern of length m

$\pi \leftarrow \text{PREFIX_FUNCTION}(P)$

$q \leftarrow 0$

FOR $i \leftarrow 1$ to n **DO**

WHILE $q > 0$ and $P[q + 1] \neq T[i]$ **DO**

$q \leftarrow \pi[q]$

END

IF $P[q + 1] == T[i]$ **THEN**

$q \leftarrow q + 1$

END

END

IF $q == m$ **THEN**

 print "Pattern found with shift", $i - m$

END

$q \leftarrow \pi[q]$

Knuth Morris Pratt (**KMP**) Algorithm

Complexity Analysis:

- Function π is computed in $O(m)$ time where as the actual string-matching routine scans n characters. So worst case running time of the algorithm is given by $O(m + n)$.

Knuth Morris Pratt (**KMP**) Algorithm

Example: Compute the prefix function for pattern ababaca

- **Solution:**

- Initially, $k = 0$, $q = 2$, $\pi[1] = 0$ and $m = 7$

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	c	a
p[i]	0						

Knuth Morris Pratt (**KMP**) Algorithm

Iteration 1 : $q = 2$, $k = 0$, k is not greater than 0, so skip while loop

- $P[k + 1] = P[1] = a$ and $P[q] = P[2] = b$
- $P[k + 1] \neq P[q]$,
- so $\pi[q] = k \rightarrow \pi[2] = 0$

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	c	a
p[i]	0	0					

Knuth Morris Pratt (**KMP**) Algorithm

Iteration 2 : $q = 3$, $k = 0$, k is not greater than 0, so skip while loop

- $P[k + 1] = P[1] = a$ and $P[q] = P[3] = a$
- $P[k + 1] = P[q]$,
- so $k = k + 1 \rightarrow k = 1$, $\pi[q] = k \rightarrow \pi[3] = 1$

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	c	a
p[i]	0	0	1				

Knuth Morris Pratt (**KMP**) Algorithm

Iteration 3 : $q = 4$, $k = 1$, k is greater than 0, so enter into while loop

- $P[k + 1] = P[2] = b$ and $P[q] = P[4] = b$, $k > 0$ but $P[k + 1] = P[q]$

// break while loop

- $P[k + 1] = P[q]$,

- so $k = k + 1 \rightarrow k = 2$, $\pi[q] = k \rightarrow \pi[4] = 2$

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	c	a
p[i]	0	0	1	2			

Knuth Morris Pratt (**KMP**) Algorithm

Iteration 4 : $q = 5$, $k = 2$, k is greater than 0, so enter into while loop

- $P[k + 1] = P[3] = a$ and $P[q] = P[5] = a$,
- $k > 0$ but $P[k + 1] = P[q]$ // break while loop
- $P[k + 1] = P[q]$, so $k = k + 1 \rightarrow k = 3$
- $\pi[q] = k \rightarrow \pi[5] = 3$

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	c	a
p[i]	0	0	1	2	3		

Knuth Morris Pratt (**KMP**) Algorithm

Iteration 5 : $q = 6$, $k = 3$, k is greater than 0, so enter into while loop

$P[k + 1] = P[4] = b$ and $P[q] = P[6] = c$,

$k > 0$ and $P[k + 1] \neq P[q]$ // Enter in while loop

$k = \pi[k] \rightarrow k = \pi[3] = 1$

Now, $P[k + 1] = P[2] = b$ and $P[q] = c$,

Still $k > 0$ and $P[k + 1] \neq P[q]$ // Enter in while loop again

$k = \pi[k] \rightarrow k = \pi[1] = 0$

As $k = 0$, While loop breaks now,

$P[k + 1] = P[1] = a$ and $P[q] = P[6] = c$

$P[k + 1] \neq P[q]$, So, $\pi[q] = k \rightarrow \pi[6] = 0$

Knuth Morris Pratt (**KMP**) Algorithm

Iteration 5

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	c	a
p[i]	0	0	1	2	3	0	

Knuth Morris Pratt (**KMP**) Algorithm

Iteration 6 : $q = 7$, $k = 0$, k is not greater than 0, so skip while loop

- $P[k + 1] = P[1] = a$ and $P[q] = P[7] = a$
- $P[k + 1] = P[q]$, so $k = k + 1 = 1$
- $\pi[q] = k \rightarrow \pi[7] = 1$

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	C	a
p[i]	0	0	1	2	3	0	1

Knuth Morris Pratt (**KMP**) Algorithm

Example: Check if pattern **P** = abababca exists in text **T** = bacbababaabcbab.

- **Solution:**
- The π function for a given pattern would be,

i	1	2	3	4	5	6	7	8
P[i]	a	b	a	b	a	b	c	a
p[i]	0	0	1	2	3	4	0	1

Knuth Morris Pratt (**KMP**) Algorithm

- The first partial match is found in the second position. The length of this partial match is 1. $\pi[1] = 0$, so we won't be able to skip ahead. We will continue matching by moving patterns right by one.

b	a	c	b	a	b	a	b	a	a	b	c	b	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a	b	a	b	c	a
---	---	---	---	---	---	---	---

Knuth Morris Pratt (**KMP**) Algorithm

- The next partial match is found in position 5. The length of this match is 5. The next character of P and T does not match. $\pi[5] = 3$, which means we get to skip ahead.

b	a	c	b	a	b	a	b	a	a	b	c	b	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a	b	a	b	c	a
---	---	---	---	---	---	---	---

Knuth Morris Pratt (**KMP**) Algorithm

- $\text{Skip} = \text{partial length} - \pi[\text{partial length}] = 5 - \pi[5] = 5 - 3 = 2$
- So we can skip two characters ahead.
- The next partial match is found in position 7. The length of this match is 3. The next character of P and T does not match. $\pi[3] = 1$, which means we get to skip ahead. The Cross indicates skipped characters..

b	a	c	b	a	b	a	b	a	a	b	c	b	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

x	x	a	b	a	b	c	a
---	---	---	---	---	---	---	---

Knuth Morris Pratt (**KMP**) Algorithm

- Skip = partial length - π [partial length] = $3 - \pi[3] = 3 - 1 = 2$
- So we can skip two characters ahead.
- Now, the pattern is longer than the remaining text, so the match does not found.

b	a	c	b	a	b	a	b	a	a	b	c	b	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

x	x	a	b	a	b	a	b	c	a
---	---	---	---	---	---	---	---	---	---

Knuth Morris Pratt (**KMP**) Algorithm

Comparison:

- If m is the length of the pattern and n is the length of the text to be searched then,

Algorithm	Pre-processing time	Matching time
Naïve string matching algorithm	0	$O((n-m)*m)$
Rabin Karp string matching algorithm	$O(m)$	Average: $O(n + m)$ Worst: $O((n-m)*m)$
Knuth-Morris-Pratt algorithm	$O(m)$	$O(n)$



SUFFIX TREES AND SUFFIX ARRAYS

Some problems

- Given a pattern $P = P[1..m]$, find all occurrences of P in a text $S = S[1..n]$
- Another problem:
 - Given two strings $S_1[1..n_1]$ and $S_2[1..n_2]$ find their longest common substring.
 - find i, j, k such that $S_1[i .. i+k-1] = S_2[j .. j+k-1]$ and k is as large as possible.
- Any solutions? How do you solve these problems (efficiently)?

Exact string matching

- Finding the pattern $P[1..m]$ in $S[1..n]$ can be solved simply with a scan of the string S in $O(m+n)$ time. However, when S is very long and we want to perform many queries, it would be desirable to have a search algorithm that could take $O(m)$ time.
- To do that we have to preprocess S . The preprocessing step is especially useful in scenarios where the text is relatively constant over time (e.g., a genome), and when search is needed for many different patterns.

Applications in Bioinformatics

- Multiple genome alignment
 - Michael Hohl et al. 2002
 - Longest common substring problem
 - Common substrings of more than two strings
- Selection of signature oligonucleotides for microarrays
 - Kaderali and Schliep, 2002
- Identification of sequence repeats
 - Kurtz and Schleiermacher, 1999

Suffix trees

- Any string of length m can be degenerated into m suffixes.
 - **a b c d e f g h** (length: 8)
 - 8 suffixes:
 - **h, gh, fgh, efgh, defgh, cdefgh, bcefg, abcdefgh**
- The suffixes can be stored in a suffix-tree and this tree can be generated in **$O(n)$** time
- A string pattern of length m can be searched in this suffix tree in **$O(m)$** time.
 - Whereas, a regular sequential search would take $O(n)$ time.

History of suffix trees

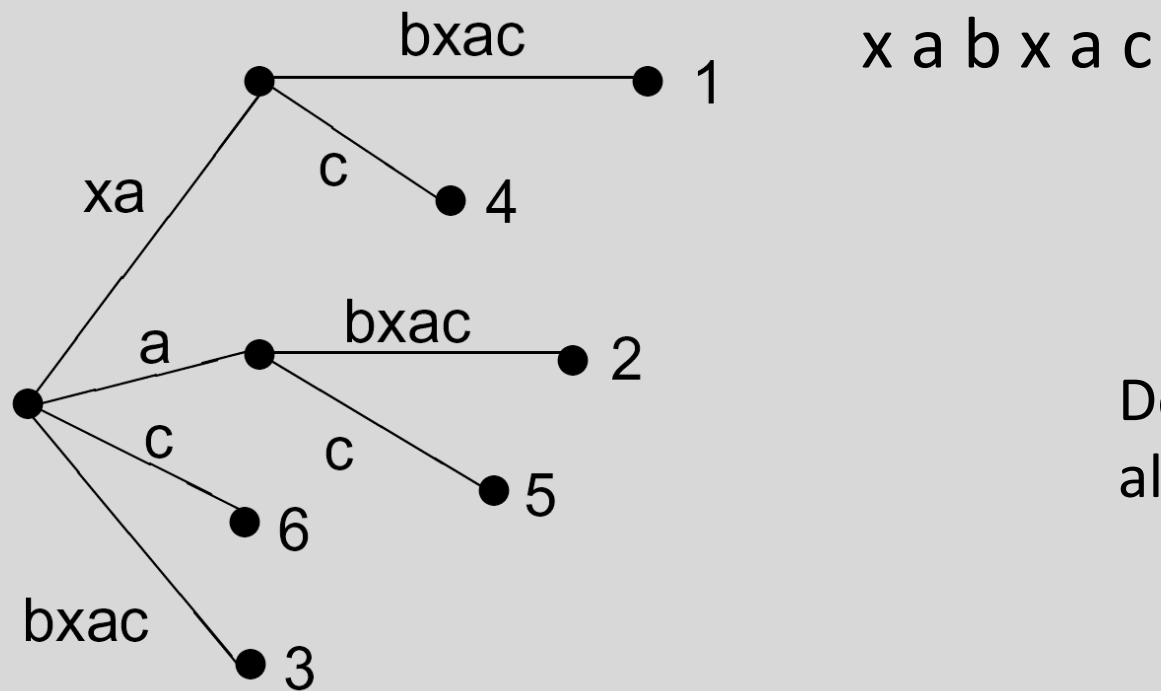
- Weiner, 1973: suffix trees introduced, linear-time construction algorithm
- McCreight, 1976: reduced space-complexity
- Ukkonen, 1995: new algorithm, easier to describe

Definition of a suffix tree

- Let $S=S[1..n]$ be a string of length n over a fixed alphabet Σ . A suffix tree for S is a tree with n leaves (representing n suffixes) and the following properties:
 - Every internal node other than the root has at least 2 children
 - Every edge is labeled with a nonempty substring of S .
 - The edges leaving a given node have labels starting with different letters.
 - The concatenation of the labels of the path from the root to leaf i spells out the i -th suffix $S[i..n]$ of S . We denote $S[i..n]$ by S_i .

An example suffix tree

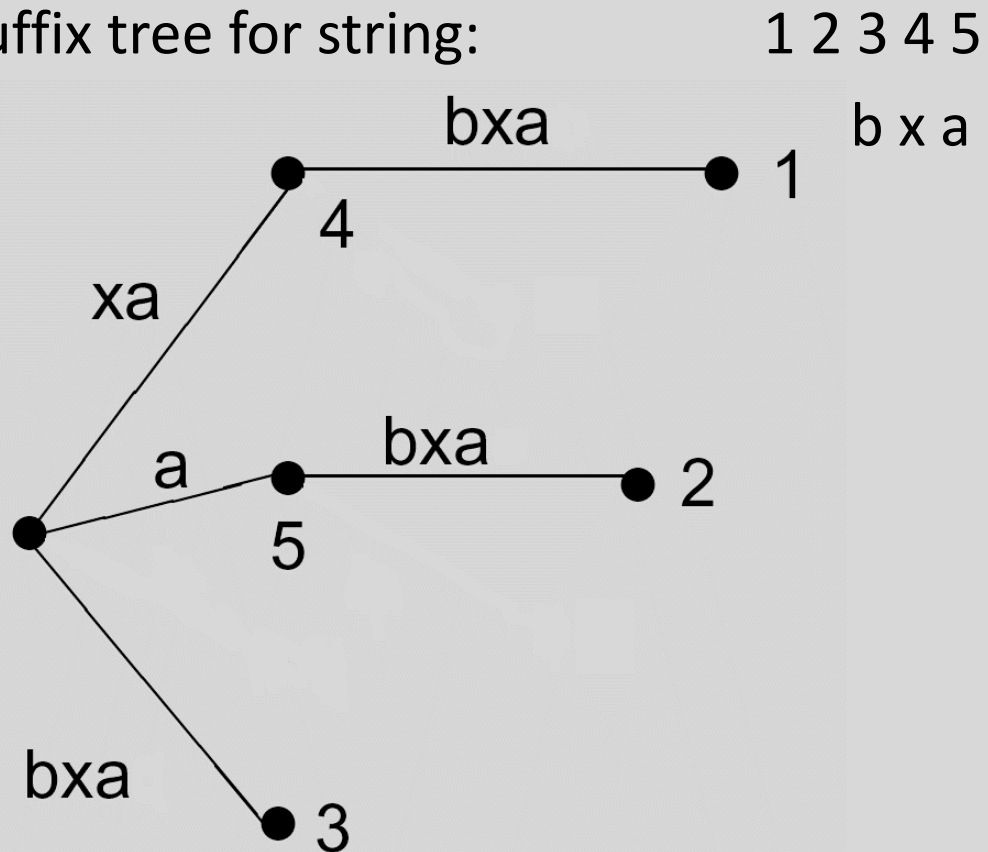
- The suffix tree for string: 1 2 3 4 5 6



Does a suffix tree
always exist?

What about the tree for xabxa?

- The suffix tree for string:



xa and *a* are not leaf nodes.

Problem

- Note that if a suffix is a prefix of another suffix we cannot have a tree with the properties defined in the previous slides.
 - e.g. *xabxa*
 - The fourth suffix *xa* or the fifth suffix *a* won't be represented by a leaf node.

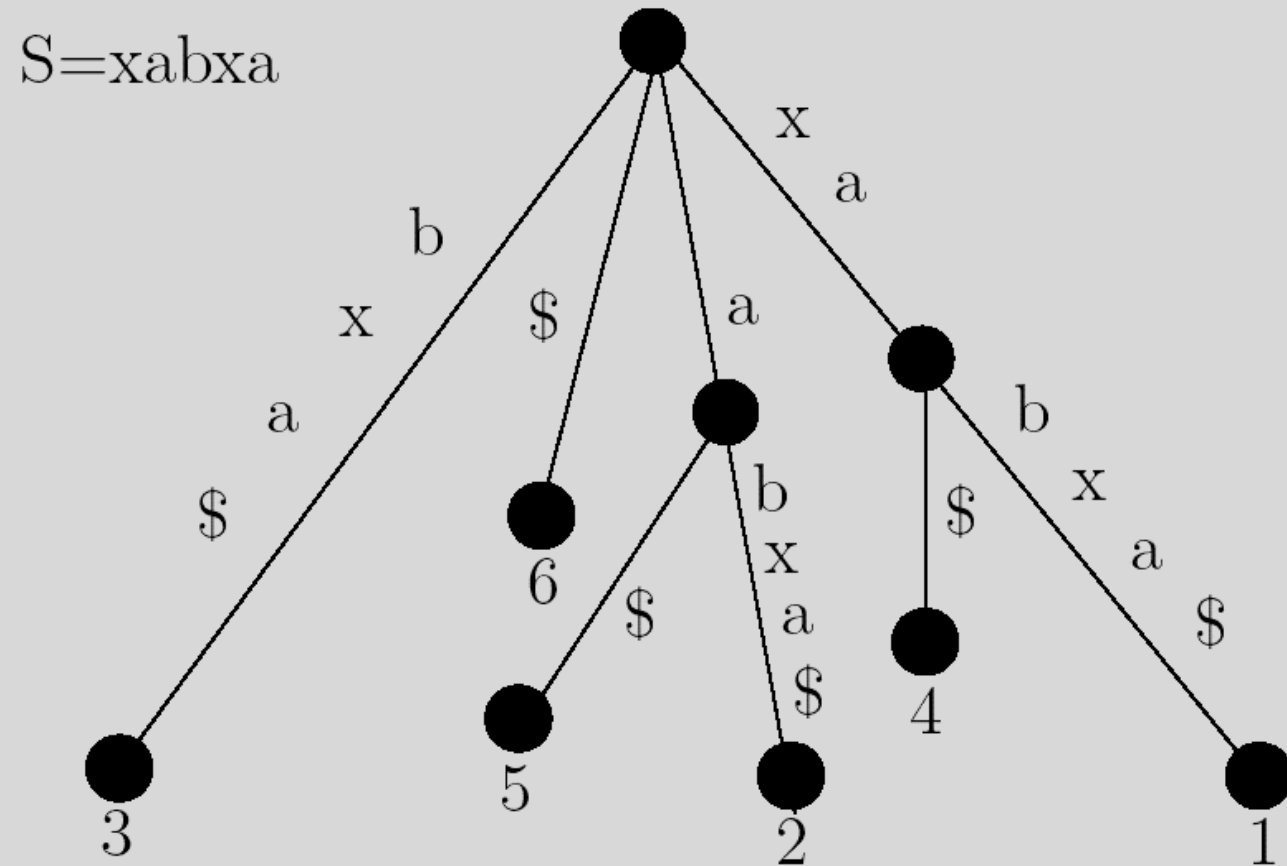
Solution: the terminal character \$

- Note that if a suffix is a prefix of another suffix we cannot have a tree with the properties defined in the previous slides.
 - e.g. *xabxa*

The fourth suffix *xa* or the fifth suffix *a* won't be represented by a leaf node.

- Solution: insert a special terminal character at the end such as *\$*.
Therefore *xa\$* will not be a prefix of the suffix *xabxa*.

The suffix tree for xabxa\$



Suffix tree construction

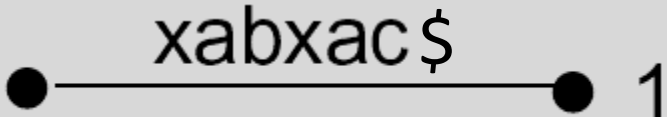
- Start with a root and **a** leaf numbered **1**, connected by an edge labeled **S** .
- Enter suffixes **$S[2..n]$** ; **$S[3..n]$** ; ... ; **$S[n]$** into the tree as follows:
- To insert **$K_i = S[i..n]$** , follow the path from the root matching characters of **K_i** until the first mismatch at character **$K_i[j]$** (which is bound to happen)
 - (a) If the matching cannot continue from a node, denote that node by **w**
 - (b) Otherwise the mismatch occurs at the middle of an edge, which has to be split

Suffix tree construction - 2

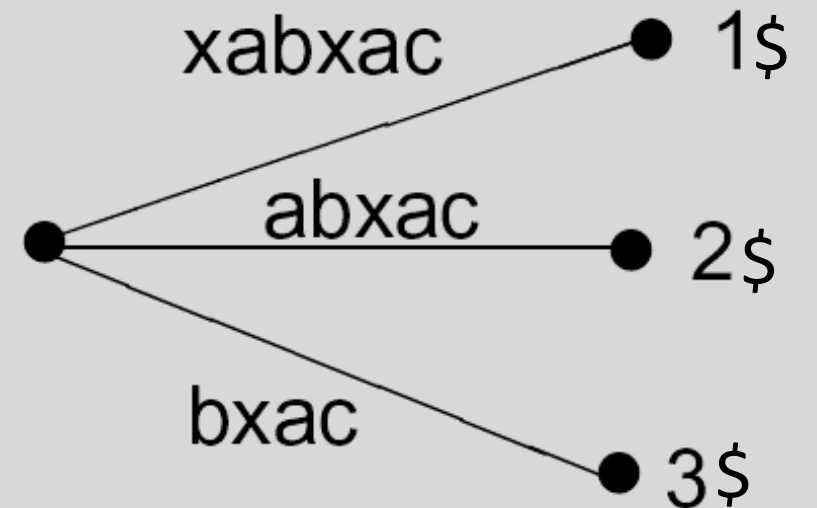
- If the mismatch occurs at the middle of an edge $e = S[u \dots v]$
 - let the label of that edge be $a_1 \dots a_l$
 - If the mismatch occurred at character a_k , then create a new node w , and replace e by two edges $S[u \dots u+k-1]$ and $S[u+k \dots v]$ labeled by $a_1 \dots a_k$ and $a_{k+1} \dots a_l$
- Finally, in both cases (a) and (b), create a new leaf numbered i , and connect w to it by an edge labeled with $K_i[j \dots |K_i|]$

Example construction

- Let's construct a suffix tree for **xabxac\$**

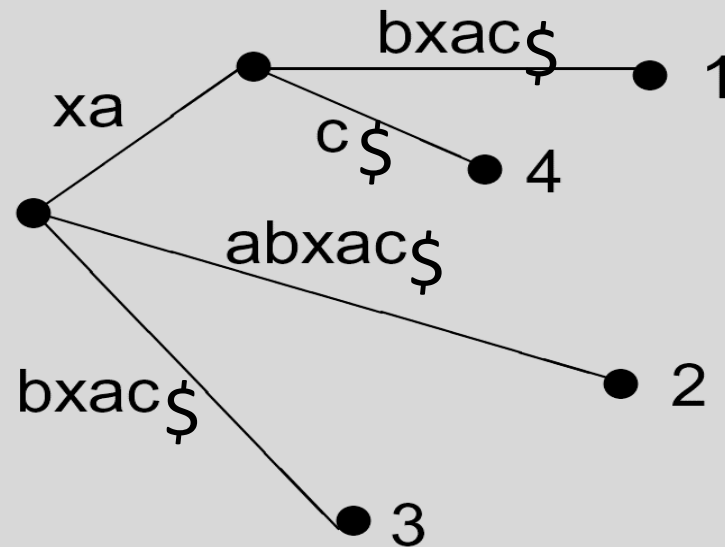
- Start with: 

- After inserting the second and third suffix:



Example contd...

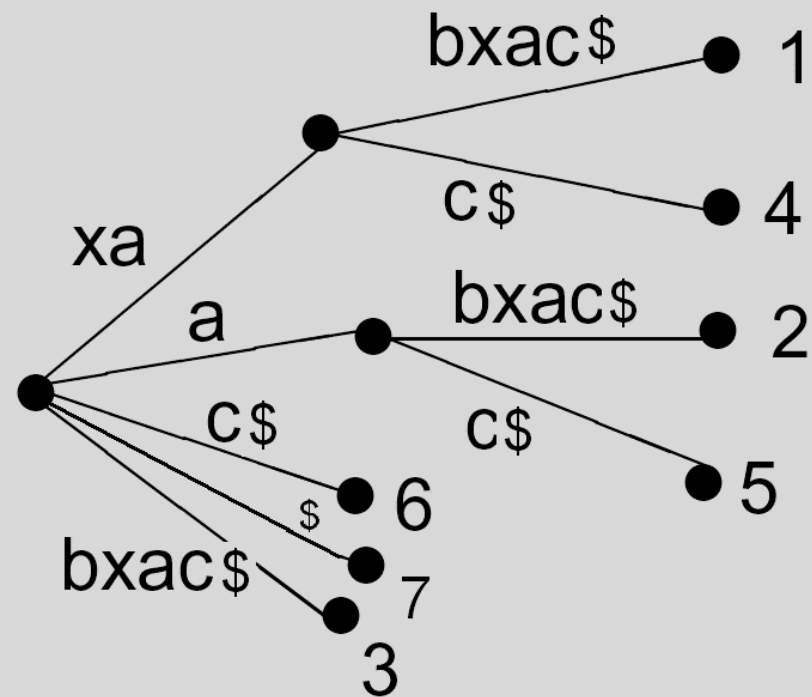
- Inserting the fourth suffix **xac\$** will cause the first edge to be split:



- Same thing happens for the second edge when **ac\$** is inserted.

Example contd...

- After inserting the remaining suffixes the tree will be completed:



Complexity of the naive construction

- We need **$O(n-i+1)$** time for the i^{th} suffix. Therefore the total running time is:

$$\sum_1^n O(i) = O(n^2)$$

- What about space complexity?
 - Can also take **$O(n^2)$** because we may need to store every suffix in the tree separately,
 - e.g., **abcdefghijklmn**

Storing the edge labels efficiently

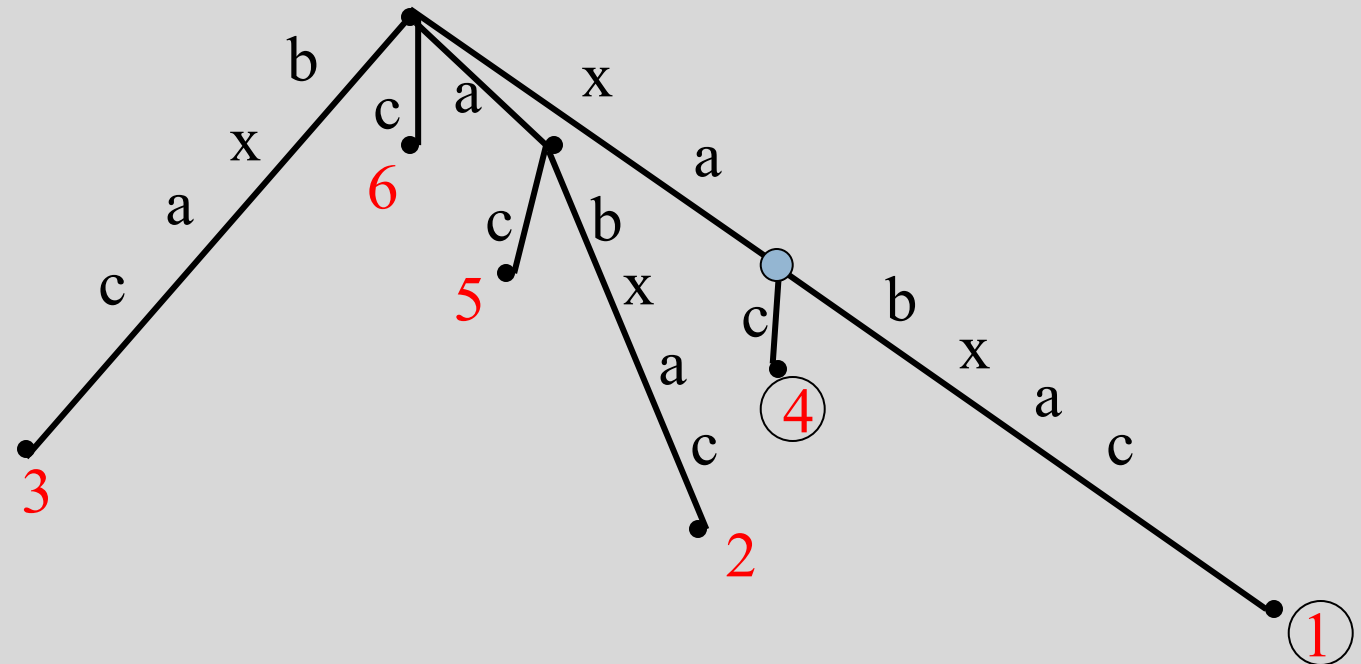
- Note that, we do not store the actual substrings $S[i \dots j]$ of S in the edges, but only their start and end indices (i, j) .
- Nevertheless we keep thinking of the edge labels as substrings of S .
- This will reduce the space complexity to $O(n)$

Using suffix trees for pattern matching

- Given S and P . How do we find all occurrences of P in S ?
- **Observation.** Each occurrence has to be a prefix of some suffix. Each such prefix corresponds to a path starting at the root.
 1. Of course, as a first step, we construct the suffix tree for S . Using the naive method this takes quadratic time, but *linear-time* algorithms (e.g., Ukkonen's algorithm) exist.
 2. Try to match P on a path, starting from the root. Three cases:
 - (a) The pattern does not match $\rightarrow P$ does not occur in T
 - (b) The match ends in a node u of the tree. Set $x = u$.
 - (c) The match ends inside an edge (v, w) of the tree. Set $x = w$.
 3. All leaves below x represent occurrences of P .

Illustration

- $T = \mathbf{xabxac}$
 - suffixes = $\{\mathbf{xabxac}, \mathbf{abxac}, \mathbf{bxac}, \mathbf{xac}, \mathbf{ac}, \mathbf{c}\}$
- Pattern P_1 : $\mathbf{xa}$
- Pattern P_2 : $\mathbf{xb}$



Running Time Analysis

- Search time:
 - $O(m+k)$ where k is the number of occurrences of P in T and m is the length of P
 - $O(m)$ to find match point if it exists
 - $O(k)$ to find all leaves below match point

Scalability

- For very large problems a linear time and space bound is not good enough. This lead to the development of structures such as Suffix Arrays to conserve memory .

Two implementation issues

- Alphabet size
- Generalizing to multiple strings

Effects of alphabet size on suffix trees

- We have generally been assuming that the trees are built in such a way that
 - from any node, we can find an edge in constant time for any specific character in Σ
 - an array of size $|\Sigma|$ at each node
- This takes $\Theta(m|\Sigma|)$ space.

More compact representation

- We may try to be more compact taking only **$O(m)$** space.
 - At each node, have pointers to only the edges that are needed
- This slows down the search time
- How much?
 - typically the minimum of **$O(\log m)$** or **$O(\log |\Sigma|)$** with a binary tree representation.
- This effects both suffix tree construction time and later searching time against the suffix tree.

Generalized suffix trees

- Build a suffix tree for a set of strings $\mathbf{S} = \{S_1, \dots, S_z\}$
- Some issues
- Nodes in tree may corresponds to substrings of potentially multiple strings S_i
 - compact edge labels: need 3 fields (start position, stop position, string)
 - leaf labels now a set of pairs indicating starting position and string

Longest common substring problem

- Build a generalized suffix tree for $S_1\$_1S_2\$_2$. Here $\$_1$ and $\$_2$ are different new symbols not occurring in S_1 and S_2 .
- Mark every internal node of the tree with $\{1\}$, $\{2\}$, or $\{1,2\}$ depending on whether its path label is a substring of S_1 and/or S_2 .
- Find the *internal* node which is labeled by $\{1,2\}$ and has the largest “**string depth**”.
- Example: (with the applet)
 - pessimist%mississippi\$

Selecting probes for microarrays

- Wikipedia: **Oligonucleotides** are short sequences of nucleotides (RNA or DNA), typically with twenty or fewer base pairs.
- Given a set of genomic sequences, the problem is to identify at least one signature oligonucleotide (probe) for each sequence. These probes must hybridize to only the desired sequence. The algorithm produces a GST from the reverse complement of all the genomic sequences (candidate probe sequences). Using the GST, the algorithm identifies all common substrings and rejects these regions because probes designed in them would not be specific to a single genomic sequence. Criteria such as probe length are used to further prune this tree.

Suffix arrays

- Suffix arrays were introduced by Manber and Myers in 1993
- More space efficient than suffix trees
- A suffix array for a string x of length m is an array of size m that specifies the lexicographic ordering of the suffixes of x .

Suffix arrays

Example of a suffix array for **acaaacatat\$**

0	aaacatat\$	3
1	aacatat\$	4
2	acaaacatat\$	1
3	acatat\$	5
4	atat\$	7
5	at\$	9
6	caaacatat\$	2
7	catat\$	6
8	tat\$	8
9	t\$	10
10	\$	11

Suffix array construction

- Naive in place construction
 - Similar to insertion sort
 - Insert all the suffixes into the array one by one making sure that the new inserted suffix is in its correct place
 - Running time complexity:
 - **$O(m^2)$** where **m** is the length of the string
- Manber and Myers give a **$O(m \log m)$** construction in their 1993 paper.

Suffix arrays

- **$O(n)$** space where n is the size of the database string
- Space efficient. However, there's an increase in query time
- Lookup query
 - Binary search
 - **$O(m \log n)$** time; m is the size of the query
 - Can reduce time to **$O(m + \log n)$** using a more efficient implementation

Searching for a pattern in Suffix Arrays

```
find(Pattern P in SuffixArray A):  
    i = 0  
    lo = 0, hi = length(A)  
    for 0 ≤ i < length(P):  
        Binary search for x, y  
        where P[i] = S[A[j] + i] for lo ≤ x ≤ j < y ≤ hi  
        lo = x, hi = y  
    return {A[lo], A[lo+1], ..., A[hi-1]}
```

Search example

- Search *is* in *mississippi\$*

Examine the pattern letter by letter, reducing the range of occurrence each time.

First letter *i*:

occurs in indices from 0 to 3

So, pattern should be between these indices.

Second letter *s*:

occurs in indices from 2 to 3

Done.

Output: *issippi\$* and *ississippi\$*

0	11	i\$
1	8	ippi\$
2	5	issippi\$
3	2	ississippi\$
4	1	mississippi\$
5	10	pi\$
6	9	ppi\$
7	7	sippi\$
8	4	sissippi\$
9	6	ssippi\$
10	3	ssissippi\$
11	12	\$

Suffix Arrays

- It can be built very fast.
- It can answer queries very fast:
 - How many times ATG appears?
- Disadvantages:
 - Can't do approximate matching
 - Hard to insert new stuff (need to rebuild the array) dynamically.

Useful links

- <http://pauillac.inria.fr/~quercia/documents-info/Luminy-98/albert/JAVA+html/SuffixTreeGrow.html>
- <http://home.in.tum.de/~maass/suffix.html>
- http://homepage.usask.ca/~ctl271/857/suffix_tree.shtml
- http://homepage.usask.ca/~ctl271/810/approximate_matching.shtml
- <http://www.cs.mcgill.ca/~cs251/OldCourses/1997/topic7/>
- <http://dogma.net/markn/articles/suffixt/suffixt.htm>
- <http://www.csse.monash.edu.au/~lloyd/tildeAlgDS/Tree/Suffix/>



MODULE 3

BCSE204L – Design and Analysis of Algorithm